

Safe Pointer Expressions in Randomly Generated C Programs

Mahesha S

Compiler Engineer

mahesha.shivamallappa@gmail.com

Abstract — Random testing of a compiler is a process in which a *randomizer* – a tool that generates randomized programs – is used to stress test the compiler with these programs. A basic assumption underlying random compiler testing is that the generated programs strictly conform to the target language’s specification, both syntactically and semantically. For random testing of a C compiler, pointer expressions generated within the random program must be safe, in the sense that they never cause out-of-bound or invalid memory access. One effective way to guarantee this safety is to incorporate pointer analysis algorithms within the C randomizer itself. This paper presents a pointer range analysis algorithm that can be incorporated within any C randomizer to ensure the safety of the pointer expressions it generates. The proposed algorithm is based on lattice theoretic iterative data flow analysis.

1. INTRODUCTION

Testing programming language compilers has been a tedious software engineering task ever since compilers were first developed [1]. A compiler typically consists of two parts: the front-end and the back-end. The front-end checks the input source program against the language specification – its syntax and semantics – while translating it into an intermediate representation. The back-end then translates this intermediate representation into a semantically equivalent, optimized object program for the target machine [2, 3, 4]. Testing a compiler therefore requires validating both parts: the front-end against the language specification, and the back-end against preservation of the input program’s semantics in the generated object code.

Compiler writers have traditionally relied on the following practical methods to test their compilers.

- Testing using plenty of manually written source programs
- Testing using commercial compiler compliance validation suites
- Regression testing

Testing with manually written source programs is the most direct of the three: compiler engineers hand-craft test cases that exercise specific language constructs, boundary conditions, and combinations of features known to be error-prone, such as nested expressions, implicit type conversions, and pointer arithmetic. This approach is precise and easy to reason about, but its coverage is bounded by what the tester can anticipate, and it scales poorly against the combinatorial space of programs a compiler must handle correctly. Commercial compiler compliance validation suites address this to some extent by offering broad, standardized coverage of a language specification, since they are built specifically to check conformance against every rule the standard defines; being fixed artifacts, however, they test only what the standard already anticipates, and are not designed to catch defects introduced by a compiler’s own optimizations or by interactions the standard does not foresee. Regression testing, in turn, is reactive rather than exploratory: every bug once

found and fixed is captured as a test case and folded permanently into the suite, guarding against its reintroduction, but such a suite only ever grows in response to bugs already discovered, offering no assurance about the far larger space of programs that have never been tried.

In recent years, an additional method known as random testing has gained attention. Here, a *randomizer* – a randomized program generation tool – stress tests the compiler with automatically generated programs. It typically takes the language specification as input, randomly traverses the language’s grammar tree, and constructs a program that is syntactically and semantically valid by preserving the grammar’s rules throughout the traversal.

When a C randomizer generates pointer expressions, it must ensure they cannot cause out-of-bound or invalid memory access. Doing so requires tracking the state of every pointer variable at each program point, which in turn requires the randomizer to incorporate an appropriate pointer analysis. This paper presents such an algorithm: a pointer range analysis that any C randomizer can incorporate to guarantee the safety of the pointer expressions it generates.

The remainder of the paper is organized as follows. Section 2 gives an overview of pointer range analysis. Section 3 surveys related work. Section 4 presents the proposed algorithm, and Section 5 illustrates it through a case study. Section 6 concludes the paper.

2. POINTER RANGE ANALYSIS

Pointer range analysis is a static program analysis technique that computes a *safe* approximation of the set of memory locations a pointer may reference at a given program point [5, 6]. Here, ‘safe’ means that a pointer expression generated using this approximation will never cause the underlying pointer to point to an out-of-bound or otherwise improper memory location. The usefulness of pointer range analysis is manifold: within a compiler, it enables more aggressive optimizations and better object code; in a static analysis tool, it allows possible run-time memory violations to be reported ahead of execution; and in a randomized C program generator, it enables the generation of safe pointer expressions. Pointer range analysis is also valuable for program verification, debugging, and program comprehension more broadly.

3. RELATED WORK

A substantial body of research exists on random testing. Since this paper focuses specifically on generating safe pointer expressions, we do not survey that literature here; the interested reader can find further references on random testing techniques in [7].

Pointer analysis, by contrast, has been an active research area for over four decades, dating back to early treatments of pointer and alias effects in interprocedural data flow analysis [8]. Researchers in program analysis, compiler construction, and software engineering more broadly have produced a large body of work since, including numerous PhD theses, yet the field remains active precisely because the underlying problems are undecidable in general. Readers new to the area can consult [9, 10, 11] for an introduction. Anderson describes a context-insensitive, flow-insensitive pointer analysis algorithm [12], while Tok et al. [13] and Hardekopf and Lin [14] present flow-sensitive alternatives. Lattner et al. [15] describe a context-sensitive, field-sensitive algorithm that incorporates heap cloning and show that it scales to practical workloads. Lin-Lojo et al. [16, 17] describe methods for parallelizing inclusion-based points-to analysis on a GPU. Closer to the present work, His and Horwitz [5] model pointer range analysis as an iterative data flow analysis problem.

4. PROPOSED ALGORITHM

This section presents our proposed pointer range analysis technique.

4.1. AIM

The aim of the proposed technique is to be incorporated within a randomized C program generation tool, so that the C randomizer can safely emit pointer expressions while constructing test programs used to stress test a C compiler.

4.2. DESCRIPTION

The proposed algorithm extends the one in [5] and is based on lattice theoretic iterative data flow analysis [18, 19, 20]. We formulate pointer range analysis as a forward iterative data flow problem: at each program point p , a mapping is maintained from every generated pointer variable pt to an abstract representation of the memory range pt may reference at runtime. This abstraction is safe in the sense that substituting it for the pointer's exact runtime range never alters the program's semantics.

4.3. LATTICES

This section defines the lattices that underlie our lattice theoretic, iterative data flow formulation of pointer range analysis [21, 22].

4.3.1. Integer Interval Lattice

We define the integer interval lattice as follows.

- Let $Z^\wedge = Z \cup \{-\infty, +\infty\}$, and let

$$I = \{[\min, \max] \mid \min, \max \in Z^\wedge \wedge \min \leq \max\}$$

where an element $[\min, \max] \in I$ represents the integer value range from $\min$ to $\max$.

- Define a partial order relation $\leq_I$ on I as follows.

$$[\min_1, \max_1] \leq_I [\min_2, \max_2] \quad \text{iff} \quad [\min_1, \max_1] \supseteq [\min_2, \max_2],$$

that is, $\min_1 \leq \min_2$ and $\max_1 \geq \max_2$. In this case, the system $I_D = (I, \supseteq)$ forms a lattice with top element $\top = [0, 0]$, bottom element $\perp = [-\infty, +\infty]$, and meet operator $\wedge = \cup$, where the meet operation is defined as follows.

$$[\min_1, \max_1] \cup [\min_2, \max_2] = [\min(\min_1, \min_2), \max(\max_1, \max_2)]$$

4.3.2. Array Descriptor Lattice

We define the array descriptor lattice as follows.

- Let $L = \{v \mid v \text{ is a non-pointer variable}\} \cup \{\text{unknown}\}$ where unknown represents some abstract unknown location. First, every element of L is treated as a single dimensional array with minimum array size set as 1. Next, a relation $\leq_L$ on L is defined as follows. $\forall x, y \in L, x \leq_L y$ iff $x = y$ (iff x and y are aliases). Further, we assume $\forall x \in L, \text{unknown} \leq x$.
- Let T be a set of unqualified, non-void, non-array C types, and let N be a set of natural numbers. Define $D = L \times T \times N$, where an element $\langle x, \tau, \sigma \rangle \in D$ represents an array location x whose size is σ , and whose elements' type is τ . For readability, we use the notation $x : \tau[\sigma]$ to represent the

triple $\langle x, \tau, \sigma \rangle \in D$. The element $\text{unknown} : \tau[\sigma]$ represents an array location whose size is σ , whose elements' type is τ , but whose initial address is not known. A relation $\leq_D$ is defined on D as follows.

$$x_1:\tau_1[\sigma_1] \leq_D x_2:\tau_2[\sigma_2] \quad \text{iff} \quad x_1 \leq_L x_2 \wedge \tau_1 = \tau_2 \wedge \sigma_1 \leq \sigma_2.$$

An operation $\cup$ on D w.r.t. $\leq_D$ is defined as follows.

$$x_1:\tau_1[\sigma_1] \cup x_2:\tau_2[\sigma_2] = x:\tau[\sigma]$$

where x is either x_1 or x_2 if $x_1 = x_2$, or unknown otherwise; $\tau = \tau_1 = \tau_2$; and finally $\sigma = \min(\sigma_1, \sigma_2)$.

- Define $DI = \{D \cup \{\text{NULL}\}\} \times I$, where an element $\langle x:\tau[\sigma], [\min, \max] \rangle \in DI$ represents an array location x whose size is σ , whose elements' type is τ , and whose array index offset is in the range $[\min, \max]$. A relation $\leq_{DI}$ is defined on DI as follows.

$$(d_1, i_1) \leq_{DI} (d_2, i_2) \quad \text{iff} \quad d_1 \leq_D d_2 \wedge i_1 \leq_I i_2.$$

In this case, the system $(DI, \leq_{DI})$ forms a lattice with top element $\top = \langle \text{NULL}, [0, 0] \rangle$, bottom element $\perp = \langle \text{NULL}, [-\infty, +\infty] \rangle$, and the meet operation on DI is defined as the concatenation of the meet operations of D and I .

4.4. IDEA

The idea behind our approach is as follows. At each program point p , we associate every pointer variable pt with an element $\langle x:\tau[\sigma], [\min, \max] \rangle$ from DI , which we call the *points-to-description* of pt at p . This description tells us that pt is tied to an array location x of σ elements of type τ , and that pt points somewhere within the range $[x[\min], x[\max]]$. We use this information to generate a safe pointer arithmetic expression for pt at p in the C randomizer's output. The natural question then is – when a points-to-description is *safe* for pointer arithmetic operations on pt – a notion we define formally below.

4.4.1. Safe Points-to-Description for Pointer-Integer Operations

Let p be an arbitrary program point and pt an arbitrary pointer variable in the program, with points-to-description $(pt, \langle x:\tau[\sigma], [\min, \max] \rangle)$ at p . We say this points-to-description of pt at p is *safe*

- if $i < \min < \max \leq \sigma - 1$ for the pointer operations of the type $pt - i$, where i is some positive integer
- if $i \leq \min < \max < \sigma - 1$ for the pointer operations of the type $pt + i$ or $i + pt$, where i is some positive integer

4.4.2. Safe Points-to-Description for Pointer-Pointer Operations

Let p be an arbitrary program point, and pt_1, pt_2 two arbitrary pointer variables in the program, with points-to-descriptions

$$(pt_1, \langle x_1:\tau_1[\sigma_1], [\min_1, \max_1] \rangle) \text{ and } (pt_2, \langle x_2:\tau_2[\sigma_2], [\min_2, \max_2] \rangle)$$

at p . These descriptions are safe for the pointer-pointer operations $pt_2 - pt_1$ or $pt_1 - pt_2$ at p if

- $x_1:\tau_1[\sigma_1] = x_2:\tau_2[\sigma_2]$, and
- $x_1 \neq \text{unknown}$ and $x_2 \neq \text{unknown}$, and
- $0 \leq \min_1 \leq \max_1 \leq \sigma_1 - 1$ and $0 \leq \min_2 \leq \max_2 \leq \sigma_2 - 1$

4.5. DATA FLOW FRAMEWORK

As mentioned in Section 4.2, we model pointer range analysis as a forward iterative data flow problem and define the corresponding data flow framework as follows [22]. Let $G = \langle N, E, n_s, n_e \rangle$ be the control flow graph (CFG) modeling the program P under consideration, where N is its set of nodes, E its set of directed edges, $n_s \in N$ its start node, and $n_e \in N$ its end node.

The data flow framework $DFA_G = \langle (L, \leq)_G, \wedge_G, F_G, d_G \rangle$ – comprising the data flow lattice $(L, \leq)_G$, the meet operator $\wedge_G$, the set of transfer functions F_G , and the analysis direction d_G – is constructed for our pointer range analysis as follows.

- $(L, \leq)_G = (DI, \leq_{DI})$ with top element

$$\top = \langle \text{NULL}, [0, 0] \rangle \quad \text{and bottom element} \quad \perp = \langle \text{NULL}, [-\infty, +\infty] \rangle,$$

where NULL indicates the null pointer location.

- The meet operator $\wedge_G = \prod$ is defined as below.

$$\prod_{k \in N} \langle d_k, i_k \rangle = \begin{cases} \langle \text{NULL}, [0, 0] \rangle & \text{if any } d_k = \text{NULL} \\ \langle \text{meet}(d_k), \text{meet}(i_k) \rangle & \text{otherwise} \end{cases}$$

- The transfer functions $f_n : L \rightarrow L \in F_G$, defined at the entry and exit of each node $n \in N$, are defined as below.

$$\langle d, i \rangle_{(n, in)} = \begin{cases} \top & \text{for } n = n_s \\ \prod_{k \in \text{Pred}(n)} \langle d_k, i_k \rangle & \text{otherwise} \end{cases}$$

$$\langle d, i \rangle_{(n, out)} = \begin{cases} \langle d, i \rangle_{(n, new)} & \text{if pointer is updated at } n \\ \langle d, i \rangle_{(n, in)} & \text{otherwise} \end{cases}$$

- The direction of data flow analysis d_G is forward.

4.6. FORMAL DESCRIPTION OF ALGORITHM

We formally describe the iterative data flow analysis algorithm below.

Algorithm 1: Forward Iterative Pointer Range Analysis

Input:

- 1: An instance of data flow framework $DFA_G = ((L, \leq)_G, \wedge_G, F_G, d_G)$ of a control flow graph $G = (N, E, n_s, n_e)$
- 2: An initial boundary value $BI = \langle \text{NULL}, [0, 0] \rangle$

Output:

- 3: Points-to-description values $(pt, \langle d, i \rangle_{(n, in)})$ and $(pt, \langle d, i \rangle_{(n, out)})$ for each pointer variable pt and for each node $n \in N$

Method:

- 4: **for** each pointer variable pt **do**

```

5:  for each node  $n \in N$  do
6:     $\langle d, i \rangle_{(n, out)} = \text{BI}$ 
7:  end for
8:   $\text{WorkList} = N - \{n_s\}$ 
9:  while  $\text{WorkList} \neq \emptyset$  do
10:    remove a node  $n$  from  $\text{WorkList}$ 
11:     $\langle d, i \rangle_{(n, in)} = \prod_{k \in \text{Pred}(n)} \langle d_k, i_k \rangle$ 
12:     $\langle d, i \rangle = \langle d, i \rangle_{(n, out)}$ 
13:     $\langle d, i \rangle_{(n, out)} = \begin{cases} \langle d, i \rangle_{(n, new)} & \text{if } pt \text{ updated at } n \\ \langle d, i \rangle_{(n, in)} & \text{otherwise} \end{cases}$ 
14:    if  $\langle d, i \rangle \neq \langle d, i \rangle_{(n, out)}$  then
15:       $\text{WorkList} = \text{WorkList} \cup \text{succ}(n)$ 
16:    end if
17:  end while
18: end for

```

5. A CASE STUDY

Fig. 1 shows a small example program, and Fig. 2 shows its control flow graph, which we use as a case study for the algorithm described in Section 4. In Fig. 2, the entry and exit of each node are annotated with the points-to-description of the pointer variable p .

```

int a[10], b[8];
int *p;

if (...) {
  p = &a[6];
} else {
  p = &b[3];
}
p++;

```

Figure 1: C program segment

At the entry of node $N1$, the points-to-description of p is set to the top element of the array descriptor lattice, $\langle \text{null}, [0, 0] \rangle$. Since $N1$ makes no new assignment to p , this points-to-description propagates unchanged from the entry to the exit of $N1$, and from there to the entry of both $N2$ and $N3$. Both nodes assign new pointer values to p , so their transfer functions update the points-to-description of p at their respective exits to $(p, \langle a:\text{int}[10], [6, 6] \rangle)$ and $(p, \langle b:\text{int}[8], [3, 3] \rangle)$.

At the entry of $N4$, merging the incoming points-to-descriptions

$$(p, \langle a:\text{int}[10], [6, 6] \rangle) \text{ and } (p, \langle b:\text{int}[8], [3, 3] \rangle)$$

yields $(p, \langle \text{unknown}:\text{int}[8], [3, 6] \rangle)$. This tells us that although the exact location p points to is no longer known, its safe type is an array of 8 elements of type int , and p is guaranteed to point somewhere within $[3, 6]$. This is enough to conclude that the pointer increment at $N4$ is safe; the transfer function accordingly updates the points-to-description at the exit of $N4$ to $(p, \langle \text{unknown}:\text{int}[8], [4, 6] \rangle)$.

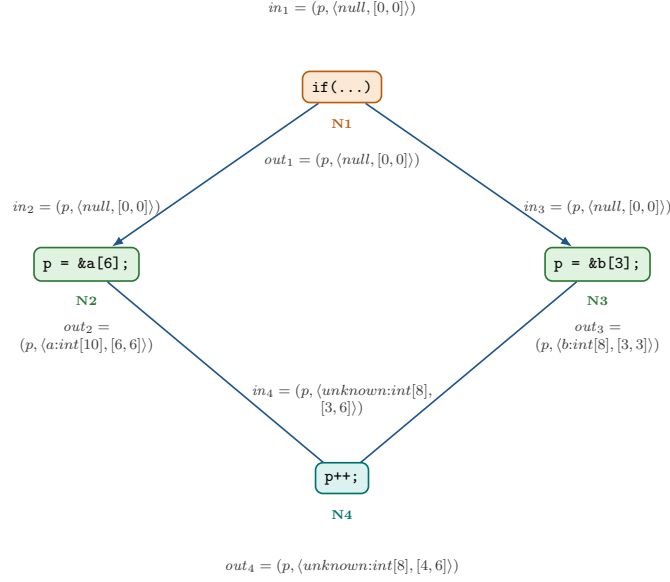


Figure 2: Control flow graph representing the C program segment of Fig. 1. Amber marks the branch node, green the pointer-assignment nodes, and teal the pointer-arithmetic node. The italicized text above and below each node gives the data flow value (points-to-description of p) just before control enters, and just after control leaves, that node.

6. CONCLUSIONS

We have presented a lattice theoretic, iterative data flow analysis algorithm for pointer range analysis, extending the algorithm of [5] to aid in generating safe pointer arithmetic expressions for compiler stress testing. We implemented it within a randomizer framework for generating C test programs, and used its analysis results to emit safe pointer arithmetic expressions at different program points of the randomly generated programs.

REFERENCES

- [1] A. P. Mathur. *Foundations of Software Testing*. Dorling Kindersley India Pvt Ltd, 2012.
- [2] A. V. Aho, R. Sethi, and J. D. Ullman. *Compiler Principles, Techniques and Tools*. Pearson Education Pvt Ltd, Indian Branch, Delhi, 2004.
- [3] S. S. Muchnick. *Advanced Compiler Design and Implementation*. Replika Press Pvt Ltd, Delhi, 1997.
- [4] A. W. Appel. *Modern Compiler Implementation in C*. Replika Press Pvt Ltd, Delhi, 2003.
- [5] S. His and S. Horwitz. Pointer-range analysis. In *Static Analysis Symposium (SAS 2004)*, volume 3148 of *LNCS*, pages 133–148, 2004.
- [6] R. Rugina. Symbolic bounds analysis of pointers, array indices, and accessed memory regions. *ACM Transactions on Programming Languages and Systems*, 27(4):185–235, 2005.
- [7] C. Yang et al. Taming compiler fuzzers. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Seattle, WA, 2013.

- [8] W. E. Weihl. Interprocedural data flow analysis in the presence of pointers, procedure variables, and label variables. In *POPL '80: Proceedings of the 7th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, Las Vegas, NV, 1980.
- [9] M. Hind. Pointer analysis: Haven't we solved this problem yet? In *PASTE'01*, Snowbird, Utah, USA, 2001.
- [10] M. Hind and A. Pioli. Which pointer analysis should i use? In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2000)*, Portland, Oregon, 2000.
- [11] M. Hind, M. Burke, P. Carini, and J. Choi. Interprocedural pointer alias analysis. *ACM Transactions on Programming Languages and Systems*, 21(4):848–894, 1999.
- [12] L. O. Anderson. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994.
- [13] T. B. Tok, S. Z. Guyer, and C. Lin. Efficient flow-sensitive interprocedural data-flow analysis in the presence of pointers. In *Compiler Construction (CC 2006)*, volume 3923 of *LNCS*, pages 17–31, 2006.
- [14] B. Hardekopf and C. Lin. Flow-sensitive pointer analysis for millions of lines of code. In *9th Annual IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 289–298, 2011.
- [15] C. Lattner, A. Lenharth, and V. Adve. Making context-sensitive points-to analysis with heap cloning practical for the real world. In *PLDI'07*, San Diego, California, USA, 2007.
- [16] M. Lin-Lojo, M. Mathew, and K. Pingali. Parallel inclusion-based points-to analysis. In *OOPSLA/SPLASH'10*, Reno/Tahoe, Nevada, USA, 2010.
- [17] M. Lin-Lojo, M. Burtcher, and K. Pingali. A gpu implementation of inclusion-based points-to analysis. In *PPoPP'12*, New Orleans, Louisiana, USA, 2012.
- [18] G. A. Kildall. A unified approach to global program optimization. In *POPL '73: Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 194–206, 1973.
- [19] P. Cousot and R. Cousot. Static determination of dynamic properties of generalized type unions. In *Proceedings of an ACM Conference on Language Design for Reliable Software*, pages 77–94, 1977.
- [20] P. Cousot and R. Cousot. Static determination of dynamic properties of programs. In *Proceedings of the 2nd International Symposium on Programming*, Paris, 1976.
- [21] K. S. Larsen. A note on lattices and fixed points. Technical report, Department of Mathematics and Computer Science, University of Southern Denmark, 2007.
- [22] U. P. Khedker, A. Sanyal, and B. Karkare. *Data Flow Analysis: Theory and Practice*. Taylor & Francis Group, Boca Raton, FL, 2009.